

Лабораторная работа №1

Julia. Установка и настройка. Основные принципы.

1. Цель работы

Подготовить рабочее пространство и инструментарий для работы с языком программирования Julia, на простейших примерах познакомиться с основами синтаксиса Julia.

2. Задание

1. Установить под свою операционную систему Julia, Jupyter
 2. Используя Jupyter Lab, повторить примеры из лабораторной работы
 3. Выполнить задания для самостоятельной работы
-

3. Выполнение работы

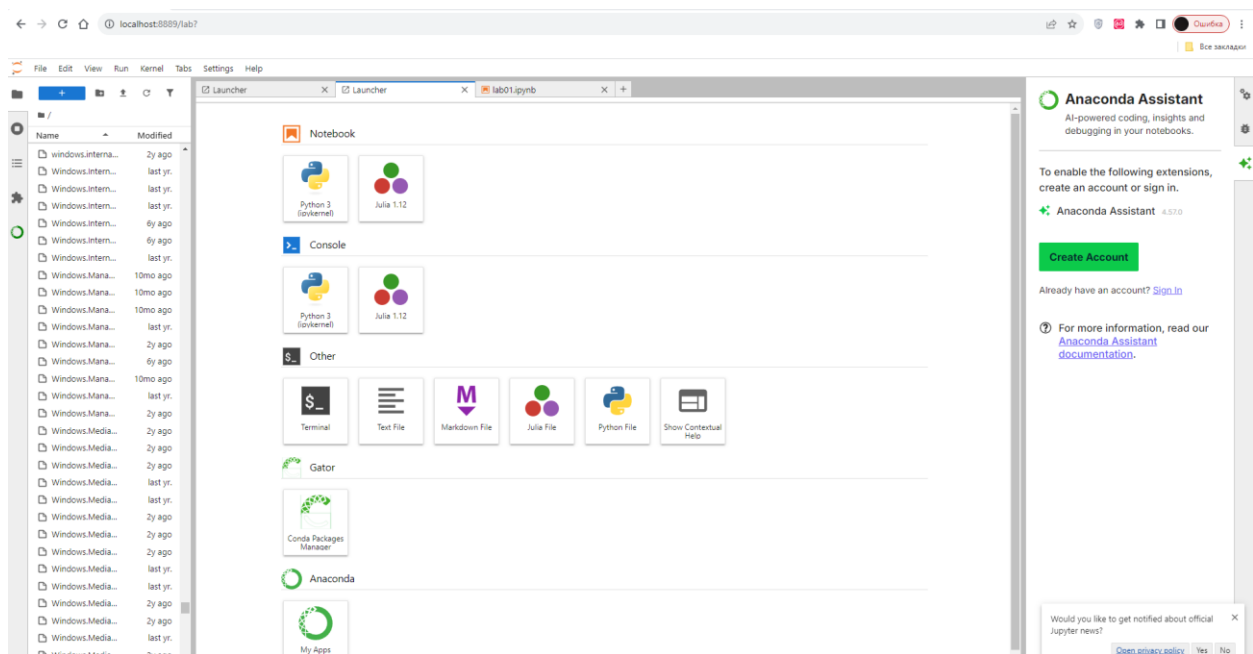
3.1 Установка Julia

Была установлена Julia версии 1.12.0.

```
PS C:\Windows\system32> julia --version
julia version 1.12.0
PS C:\Windows\system32> 
```

3.2 Установка Jupyter Lab

Был установлен Jupyter Lab через pip.



3.3 Установка пакетов Julia

В Julia REPL были установлены пакеты:

- IJulia
- DrWatson
- Plots
- DataFrames
- CSV
- JLD2
- Literate
- DifferentialEquations
- BenchmarkTools

```
julia> Pkg.status()
status `C:\Users\Asus\.julia\environments\v1.12\Project.toml`
[6e4b80f9] BenchmarkTools v1.8.0
[336ed68f] CSV v0.10.17
[a93c6f00] DataFrames v1.8.2
[0c46a032] DifferentialEquations v8.1.1
[634d3b9d] DrWatson v2.19.1
[7073ff75] IJulia v1.34.4
[033835bb] JLD2 v0.6.6
[98b081ad] Literate v2.21.0
[91a5bccd] Plots v1.41.7

julia>
```

3.4 Примеры из лабораторной работы

3.4.1 Определение типов числовых величин

```
[9]: typeof(3)
      typeof(3.5)
      typeof(3/3.5)
      typeof(π)
```

```
[9]: Irrational{:π}
```

3.4.2 Специальные значения

```
[11]: typeof(Inf)
      typeof(-Inf)
      typeof(NaN)
```

```
[11]: Float64
```

3.4.3 Диапазоны целочисленных типов

```
[13]: for T in [Int8, Int16, Int32, Int64, Int128, UInt8, UInt16, UInt32, UInt64, UInt128]
      println("${lpad(T, 7)}: [{typemin(T)}, {typemax(T)}]")
      end

      Int8: [-128, 127]
      Int16: [-32768, 32767]
      Int32: [-2147483648, 2147483647]
      Int64: [-9223372036854775808, 9223372036854775807]
      Int128: [-170141183460469231731687303715884105728, 170141183460469231731687303715884105727]
      UInt8: [0, 255]
      UInt16: [0, 65535]
      UInt32: [0, 4294967295]
      UInt64: [0, 18446744073709551615]
      UInt128: [0, 340282366920938463463374607431768211455]
```

3.4.4 Преобразование типов

```
[15]: Int64(2.0)
      Char(2)
      convert{Int64, 2.0}
      convert{Char, 2}
      Bool(1)
      Bool(0)
      promote{Int8(1), Float16(4.5), Float32(4.1)}
```

```
[15]: (1.0f0, 4.5f0, 4.1f0)
```

3.4.5 Определение функций

```
[17]: function f(x)
      x^2
      end
      f(4)

      g(x) = x^2
      g(8)
```

```
[17]: 64
```

3.4.6 Работа с массивами

```
[19]: a = [4 7 6]
      b = [1, 2, 3]
      a[2], b[2]

      a=1; b=2; c=3; d=4
      Am = [a b; c d]
      Am[1,1], Am[1,2], Am[2,1], Am[2,2]

[19]: (1, 2, 3, 4)
```

3.5 Самостоятельная работа

3.5.1 Функции ввода/вывода

```
[33]: print("Hello")
      print(" World")

      println("Hello")
      println("World")

      show(stdout, MIME("text/plain"), [1, 2, 3])

      Hello WorldHello
      World
      3-element Vector{Int64}:
       1
       2
       3
```

3.5.2 Функция parse()

```
[25]: parse{Int, "123"}
      parse{Float64, "3.14"}
      parse{Int, "FF", base=16}

[25]: 255
```

3.5.3 Базовые математические операции

```
[27]: # Арифметика
      2 + 3
      5 - 2
      3 * 4
      8 / 3
      2^10
      sqrt(16)

      # Сравнение
      3 > 2
      3 == 3
      3 != 4
      3 <= 3

      # Логика
      true && false
      true || false
      !true

[27]: false
```

3.5.4 Операции с матрицами и векторами

```
[31]: using LinearAlgebra

      # Определение матриц
      A = [1 2; 3 4]
      B = [5 6; 7 8]

      # Операции
      A + B
      A - B
      2 * A
      A'
      dot([1,2], [3,4])

[31]: 11
```

4. Графики

```
[5]: using Plots  
plot(rand(10), title="Мой первый график", legend=false)
```



5. Выводы

Выводы

1. Установлены Julia и Jupyter Lab на операционную систему.
2. Настроена рабочая среда для выполнения лабораторных работ.
3. Изучены основы синтаксиса Julia:
 - Определение типов данных
 - Преобразование типов
 - Создание функций
 - Работа с массивами и матрицами
 - Базовые математические операции
4. Изучены функции ввода/вывода и функция `parse()`.
5. Освоены операции с матрицами и векторами.

Julia является удобным и мощным языком для научных вычислений. Его синтаксис интуитивно понятен, особенно для пользователей, знакомых с MATLAB или Python. Встроенная поддержка математических операций и типов данных делает его отличным выбором для математического моделирования и анализа данных.